GNN-EGG: Graph Neural Network Explanations *via* Graph Generation

Art Tay

Abstract

Keywords: Graph Neural Networks, Interpretable AI

Introduction

Related Work

Methodology

Notation

Let G be a graph with X, E, and A being the node feature matrix, edge feature matrix, and adjacency matrix respectively. Let $X = [X_c, X_d]$ and $E = [E_c, E_d]$ where c and d denote continuous and one-hot discrete features.

Generation

X_c and E_c can both be sampled from any function with the form:

$$f(z\ ;\ \Omega) : \mathbb{R}^Z \to \mathbb{R}^{dim} \tag{1}$$

and *dim* denotes the dimensionality of the matrix we want to generate. z could be sampled as pure random noise, but z could also be a function of the data. In most instances, we generate continuous feature matrices using a random vector from a multivariate normal pass through a standard dense layer with no activation.

X_d and E_d can both be sampled from the *concrete distribution* (Maddison, Mnih, and Teh 2017). Using the reparameterization trick and continuous relaxation, we can sample one-hot categorical vectors based on two parameters θ_{Cat}, a trainable parameter vector of length equal to the number of categories, and τ, a hyperparameter that controls the degree of relaxation (smaller value approximate the discrete sampling better, but can result in numerical issues).

$$\text{Softmax}\left(\frac{\theta - log(-log\ \epsilon)}{\tau}\right), \quad \epsilon \sim U[0,1] \tag{2}$$

Like z, θ_{Cat} could be a function of the data and other trainable parameters. One concrete distribution would be required per discrete feature.

Similarly, the adjacency matrix can be sampled from the *binary concrete distribution*:

$$\text{sigmoid}\left(\frac{\theta + \log\ \epsilon - \log(1 - \log\ \epsilon)}{\tau}\right) \tag{3}$$

Identical to X. Wang and Shen (2023), set θ to be the same dimensionality as A, where each entry in θ is the probability that an edge exists between nodes i and j.

Pseudo-code for a generic *egg* generator is provided below.

```
generator(z_1, z_2, theta_1, theta_3, theta_3, tau_1, tau_2, tau_3){
    X_c = f(z_1; Omega_1)

    E_c = f(z_2; Omega_2)

    for each discrete node feature i do:
        X_di ~ concrete_1(theta_1i, tau_1)
        X_d = [X_d, X_di]

    for each discrete edge feature i do:
        E_di ~ concrete_1(theta_2i, tau_2)
        E_d = [E_d, E_di]

    A ~ binary_concrete(theta_3, tau_3)

    return X_c, X_d, E_c, E_d, A
}
```

Loss Function

GNN-EGG trains based on a three part loss function.

GNN Prediction & Embedding Loss

Let `explainee()` be the GNN model we want to explain. Let $g()$ represent the embedding layers and $h()$ represent the dense predictions layers. $g()$ is assumed to return a vectorization of a graph, and $h()$ is assumed to return a class probability vector.

$$\mathcal{L}_p = \text{distance}(g(G), \psi) + \text{Cross-Entropy}(h(g(G)), t) \tag{4}$$

t denotes the desired prediction, and ψ can either be the average embedding for the target class, like X. Wang and Shen (2023) or a particular observation.

Most GNN `explainee` model will require further discretization of the `generator`'s output. The typically transformation are $A' = I(A \geq 0.5)$, $X_d = \text{argmax}(X_d)$, $E_c = \text{argmax}(E_c)$; however, each one will prevent gradient backpropagation with respect to their corresponding parameters. This issue can be mitigated in two ways. First, the remaining two loss function terms do not involve the `explainee` directly and are designed to use the `generator` output without transformation [Discuss edge weights here]. Second, a REINFORCE (Williams 1992) term can be added based on the likelihood of the generating distributions. For example if the A' transformation is required, for following term can be added to ensure training of the edge probabilities.

$$\mathcal{L}_R = -\frac{l(\theta_A \mid G)}{\mathcal{L}} \tag{5}$$

Here $\mathcal{L}$ denotes the sum of all other loss terms and l denotes the log-likelihood. In short, this term moves θ_A to move the likelihood of G proportional to the inverse of the loss ie if $\mathcal{L}(G)$ is small, θ_A will be moved to increase the likelihood of G and vice-a-versa.

Regularization & Sparsity

Regularization the probability of edges existing will increase sparsity in terms of edges, and nodes if we remove any that become *isolated*. Identical to the regularization done in X. Wang and Shen (2023), we add

the following term.

$$\mathcal{L}_s = ||\theta_A||^2 + \text{softplus}(||\text{sigmoid}||_1 - B)^2 \tag{6}$$

B here is usually called the budget. It represents a ceiling on the number of edges that we desire on generated examples.

Structural Consistency Loss

Structurally consistency is the hardest to train, but arguable the most important property for an example to have. Most GNN use a message passing paradigm followed by pooling and dense prediction layers. The idea is to use the graph's structure to engineer better node features to draw predictions. This principle exposes a flaw in the generation scheme. If the generator is allowed to train the node features directly, it becomes likely that the generator will ignore the graph structure and leap to training the final features needed instead.

[A good example of this happening is that a model training only the node features, with all else random, was able to produce and incredible tight prediction distribution. This could also be done in spite of extreme regularization.]

Using embeddings in addition to the prediction can mitigate this problem; however, it is possible that embeddings could also be subject to this problem. For large graphs, there is a considerable amount of information that can be unrepresented in the latent vector. Two graphs could be very different on the graph level, but have very similar embeddings [include example?]. Furthermore, measurements of similarity of the latent space aren't readily interpretable by humans making is harder to evaluate and compare the realism of the generated data. Another proposed solution is to evaluate the reconstruction using a discriminator (De Cao and Kipf 2022). While more interpretable than metrics on the latent space, discriminators suffer from the issue of non-identifiability. In other words, a desirable misclassification rate could be the result of a poor discriminator instead of a good generator.

Our proposed solution is to use an approximation to the graph edit distance (GED). GED measures the similarity between two graphs by counting the minimum number of edits required to make the graphs isomorphic (or subgraph isomorphic). This has the advantage of being human interpretable. [Ex: Being able to say that 10 out of 100 nodes and edges need to be edit for the graphs to be identical (ie the graph match 90%) should be more informative than a cosine distance in the latent space for most applications.] Problematically, finding the exact optimal GED is a known NP-complete problem, which makes it infeasible for large graph or complex training algorithms (Bougleux et al. 2015). Furthermore, most exact algorithm aren't differentiable, which would necessitate slower optimization methods. Fortunately, approximating GED as a quadratic assignment problem (QAP) solves before the complexity and derivative issue (R. Wang et al. 2024).

Results

Notes/Ideas/Future Directions

Graph Sub-Sampling Methods

"GraphSAINT follows the design philosophy of directly sampling the training graph G, rather than the corresponding GCN. Our goals are to 1. extract appropriately connected subgraphs so that little information is lost when propagating within the subgraphs, and 2. combine information of many subgraphs together so that the training process overall learns good representation of the full graph" (Zeng et al. 2020).

Graph Sub-Sampling + Inverse Weighting

Let $[G_1, \dots, G_n]$ be the set of observed graphs. Let $G_{s(i,j)}$ denote the i^{th} sub-sample of the j^{th} observed graph. Let G_{gk} represent the k^{th} generated graph, where $k \in [1 \dots i+j]$. Let explainee$(G) = \rho,\ h$ denote the prediction and embedding vectors respectively. Let ρ_t denote the prediction vector of the target class and ρ_u denote an *uninformative* prediction.

$$\gamma = \text{CrtEnt}(\rho_t,\ \rho_u) \geq 0 \tag{7}$$

$$\omega = \frac{\lambda_1}{\gamma - \text{CrtEnt}(\rho_t,\ \rho_{s(i,j)}) + \epsilon} \tag{8}$$

$$\mathcal{L}_{\text{dist}} = \frac{1}{i+j} \sum_{i=1}^{n} \sum_{j=1}^{m} \text{dist}(h_{s(i,j)},\ h_{gi+j}) + \text{match}(G_{s(i,j)},\ G_{gi+j}) \tag{9}$$

- If the sub-graph prediction $\rho_{s(i,j)}$ is better than the uninformative prediction, then

$$\text{CrtEnt}(\rho_t, \rho_{s(i,j)}) < \gamma \Rightarrow \omega \geq 0 \Rightarrow \text{argmin } \mathcal{L}_{\text{dist}}$$

- If the sub-graph prediction is worse than the uninformative prediction, then

$$\text{CrtEnt}(\rho_t, \rho_{s(i,j)}) > \gamma \Rightarrow \omega < 0 \Rightarrow \text{argmax } \mathcal{L}_{\text{dist}}$$

Potential Advantages:

1. Solves the large graph memory issue.
2. Creates more training samples. Sub-sampling and observed graphs don't have to be split be class.
3. Weights the samples based on the models confidence.
4. Moves the generator towards good samples and away from bad samples.

Potential Disadvantages:

1. Selection bias.
2. Information loss.

Best Sub-Graph Match for Visualization

Let G be an observed graph. Let $[G_{s1} \ldots G_{sm}]$ be the set of sub-graph samples taken from graph G. Let G_g be a generated explanation graph. Then,

$$\text{match}(G, G_g) \approx \max \{\text{match}(G_g,\ G_{si}) \mid i \in [1,\ m]\}$$

Future Directions

- Generate graph with ρ_u as the target. Analyze generated graphs that the model is most unsure about ie what are the model's weaknesses.
- Tighten the generator distribution. Within a given batch select the generated graph with the best score (lowest loss so far), then add a matching loss term between all other generated graphs and the best one.

References

Bougleux, Sébastien, Luc Brun, Vincenzo Carletti, Pasquale Foggia, Benoit Gaüzère, and Mario Vento. 2015. "A Quadratic Assignment Formulation of the Graph Edit Distance," no. arXiv:1512.07494 (December). http://arxiv.org/abs/1512.07494.

De Cao, Nicola, and Thomas Kipf. 2022. "MolGAN: An Implicit Generative Model for Small Molecular Graphs," no. arXiv:1805.11973 (September). https://doi.org/10.48550/arXiv.1805.11973.

Maddison, Chris J., Andriy Mnih, and Yee Whye Teh. 2017. "The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables," no. arXiv:1611.00712 (March). https://doi.org/10.48550/arXiv.1611.00712.

Wang, Runzhong, Ziao Guo, Wenzheng Pan, Jiale Ma, Yikai Zhang, Nan Yang, Qi Liu, et al. 2024. "Pygmtools: A Python Graph Matching Toolkit." *Journal of Machine Learning Research* 25 (33): 1–7. https://jmlr.org/papers/v25/23-0572.html.

Wang, Xiaoqi, and Han-Wei Shen. 2023. "GNNInterpreter: A Probabilistic Generative Model-Level Explanation for Graph Neural Networks," no. arXiv:2209.07924 (February). https://doi.org/10.48550/arXiv.2209.07924.

Williams, Ronald J. 1992. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning."

Zeng, Hanqing, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2020. "GraphSAINT: Graph Sampling Based Inductive Learning Method," no. arXiv:1907.04931 (February). https://doi.org/10.48550/arXiv.1907.04931.